

Modelos DeepONet

CPE883 - Tópicos Especiais em Machine Learning

Vivian Rodrigues (DRE:125228569)

Introdução e Objetivo

Objetivo:

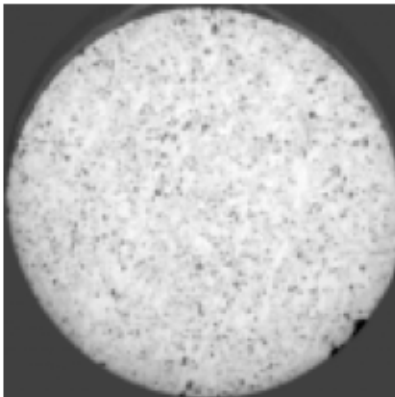
- Avaliar desempenho preliminar de redes do tipo DeepONet na segmentação de imagens de microtomografia de rochas.

Saída esperada:

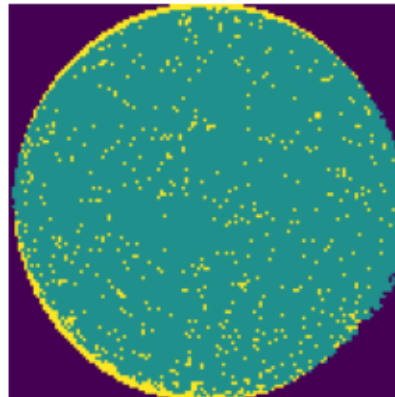
- Máscara segmentada com 3 classes (0=fundo, 1=poro, 2=rocha).

DatasetB_sampleB0033

Imagem Original



Máscara Real



Máscara Preditada



Estrutura de projeto

Objetivo:

- O pipeline foi construído de forma modular e experimental, permitindo testes controlados e rastreáveis com diferentes modelos.
- Projeto escalável e reproduzível para múltiplos modelos.

```
📁 Estrutura do Projeto
Projeto_segmentacao/
|
|— rock_seg_loader/
|   └─ rock_dataset_multi_rev2.py      # DataLoader com suporte multi-amostras e rastreabilidade
|
|— rock_seg_model/
|   └─ caps_rock_seg/                  # CapsNet
|   └─ ckan_rock_seg/                  # CKAN (KANs com convolução)
|   └─ deeponet_rock_seg/              # DeepONet (com branch e trunk convolucionais)
|
|— metrics.py                          # Métricas (Dice, IoU)
|
|— main_caps.py                        # Loop de treino para CapsNet
|— main_ckan.py                        # Loop de treino para CKAN
|— main_deeponet.py                    # Loop de treino para DeepONet
|
└─ results/
    └─ CapsNet/
    └─ CKAN/
    └─ DeepONet/
```

Dataloader – rastreabilidade de amostras

Objetivo:

Modificado para manter a rastreabilidade da origem de cada imagem e máscara.

```
# Percorre todas as subpastas (DatasetA, DatasetB, ...)
for dataset_folder in os.listdir(root_dir):
    dataset_path = os.path.join(root_dir, dataset_folder)
    input_dir = os.path.join(dataset_path, "inputs")
    mask_dir = os.path.join(dataset_path, "masks")

    if not os.path.isdir(input_dir) or not os.path.isdir(mask_dir):
        continue # Pula se não for válido

    input_files = sorted([
        f for f in os.listdir(input_dir)
        if f.endswith('.tiff') or f.endswith('.tif')
    ])

    for file_name in input_files:
        input_path = os.path.join(input_dir, file_name)
        mask_path = os.path.join(mask_dir, file_name.replace("sample", "mask"))
        if os.path.exists(mask_path):
            # Armazena também o nome da amostra (ex: "DatasetA")
            self.samples.append((input_path, mask_path, dataset_folder))
```

```
def __len__(self):
    return len(self.samples)

def __getitem__(self, idx):
    input_path, mask_path, amostra_nome = self.samples[idx] #input_path,
    mask_path, amostra_nome = self.samples[idx]
    filename = os.path.basename(input_path) # ex: "sampleA0001.tiff"

    return (
        torch.tensor(image, dtype=torch.float32),
        torch.tensor(mask, dtype=torch.long),
        f"{amostra_nome}_{filename}" # ex: "DatasetA_sampleA0001.tiff"
    )
```

Modelos

Item	CapsNet (SegCapsNet)	CKAN	DeepONetConv
Arquitetura	CapsNet adaptada para segmentação	KAN convolucional (splines aprendíveis)	DeepONet convolucional (Branch + Trunk)
Entrada	Imagem cinza (1 canal)	Imagem cinza (1 canal)	Imagem cinza (1 canal)
Nº classes	3 (fundo, rocha, poro)	3 (fundo, rocha, poro)	3 (fundo, rocha, poro)
Camadas principais	Conv2D → PrimaryCaps → Deconv → Conv final	3 × KANConv2D + Conv final	Branch Net (Conv2D) + Trunk Net (Conv2D) + FC
Ativação	Squash (cápsulas)	Splines aprendíveis (KAN)	ReLU
Fusão de features	Cápsulas (caps_dim)	Convoluções KAN	Produto escalar (branch × trunk)
Hidden Dim	–	–	64
Função de perda	CrossEntropyLoss	CrossEntropyLoss	CrossEntropyLoss
Otimizador	Adam	Adam	Adam
Resolução saída	128 × 128	64 × 64	128 × 128
Métricas	Dice Score, IoU	Dice Score, IoU	Dice Score, IoU

DeepOnet Arquitetura Detalhada

- Aplica a lógica Branch/Trunk do DeepONet com convoluções
- O branch net pode receber como entrada imagens
- O trunk net recebe coordenadas ou representações espaciais.

```
class DeepONetConv(nn.Module):
    def __init__(self, config):
        super().__init__()
        # Branch net — processa a entrada
        layers_branch = []
        in_ch = config["img_channels"]
        for out_ch, k, s in zip(config["branch_conv_channels"], config["branch_kernel_sizes"],
                                config["branch_strides"]):
            layers_branch += [nn.Conv2d(in_ch, out_ch, kernel_size=k, stride=s, padding=k//2), nn.ReLU(inplace=True)]
            in_ch = out_ch
        self.branch_net = nn.Sequential(*layers_branch)

        # Trunk net — também pode processar informação da entrada ou outra fonte
        layers_trunk = []
        in_ch = config["img_channels"]
        for out_ch, k, s in zip(config["trunk_conv_channels"], config["trunk_kernel_sizes"], config["trunk_strides"]):
            layers_trunk += [nn.Conv2d(in_ch, out_ch, kernel_size=k, stride=s, padding=k//2), nn.ReLU(inplace=True)]
            in_ch = out_ch
        self.trunk_net = nn.Sequential(*layers_trunk)
```

DeepOnet Arquitetura Detalhada

- Após a multiplicação $\text{branch} \times \text{trunk}$, o resultado é “achatado” (flatten) e passado por uma camada densa com `hidden_dim` neurônios, com ReLU.
- Essa camada serve como representação intermediária que combina as informações extraídas pelas convoluções do Branch e do Trunk, antes da projeção para a saída final de segmentação

```
# Combinação via operador (produto escalar)
hidden = config["hidden_dim"]
self.fc = nn.Sequential(
    nn.Flatten(),
    nn.Linear(in_ch * config["output_shape"][0] * config["output_shape"][1], hidden),
    nn.ReLU(inplace=True),
    nn.Linear(hidden, config["num_classes"] * config["output_shape"][0] * config["output_shape"][1])
)
self.config = config

def forward(self, x):
    b = self.branch_net(x)
    t = self.trunk_net(x)
    # junção simples — produto escalar por canal
    h = (b * t).flatten(1)
    out = self.fc(h)
    B, C, H, W = b.size(0), self.config["num_classes"], *self.config["output_shape"]
    return out.view(B, C, H, W)
```

Função de Custo, Otimização e métricas

- Função de custo: CrossEntropyLoss (multi-classe)
- Otimização: Adam(lr=1e-3)
- Batch size: 4
- Épocas: 3 (teste preliminar)
- Métricas: Dice Score, IoU

$$\text{Dice} = \frac{2 \cdot |A \cap B|}{|A| + |B|} \quad \text{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

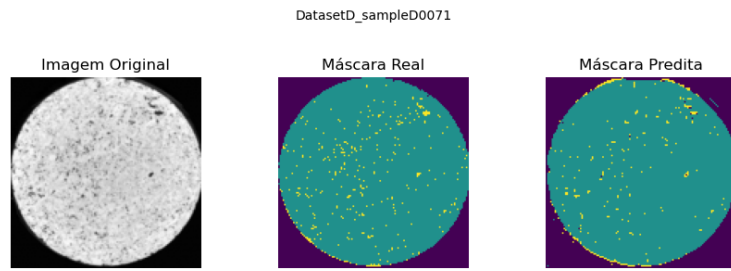
- Resolução entrada/saída: 128x128

```
# ===== Métricas =====  
def dice_score(pred, target, num_classes):  
    dice = []  
    pred = pred.cpu().numpy()  
    target = target.cpu().numpy()  
    for c in range(num_classes):  
        pred_c = (pred == c).astype(np.uint8)  
        target_c = (target == c).astype(np.uint8)  
        inter = (pred_c * target_c).sum()  
        union = pred_c.sum() + target_c.sum()  
        if union > 0:  
            dice.append(2.0 * inter / union)  
    return np.mean(dice)  
  
def iou_score(pred, target, num_classes):  
    ious = []  
    pred = pred.cpu().numpy()  
    target = target.cpu().numpy()  
    for c in range(num_classes):  
        pred_c = (pred == c).astype(np.uint8)  
        target_c = (target == c).astype(np.uint8)  
        inter = (pred_c * target_c).sum()  
        union = pred_c.sum() + target_c.sum() - inter  
        if union > 0:  
            ious.append(inter / union)  
    return np.mean(ious)
```

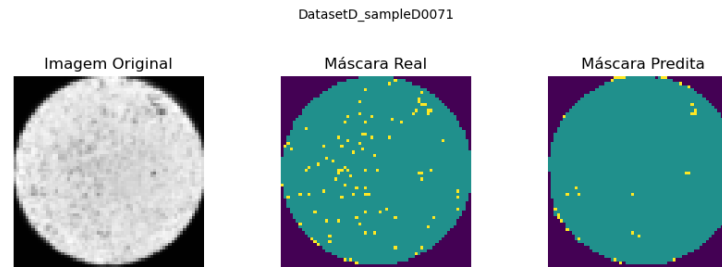

Resultados Preliminares (600 imagens)

- Imagens no conjunto de validação.
- Curva no conjunto de treinamento

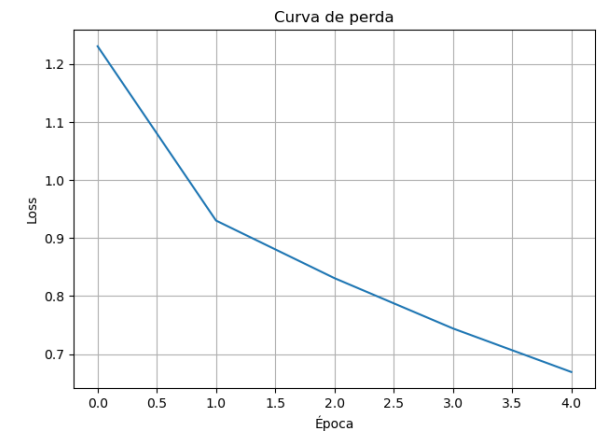
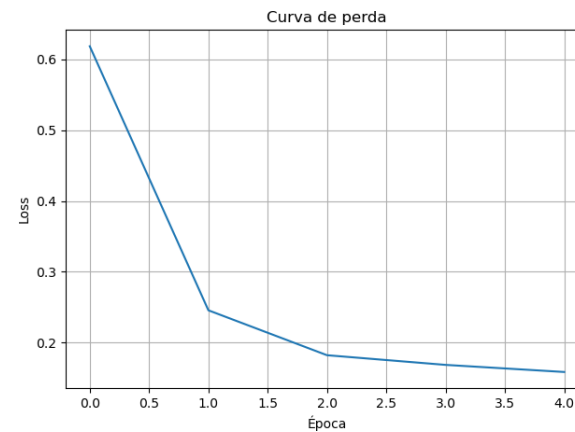
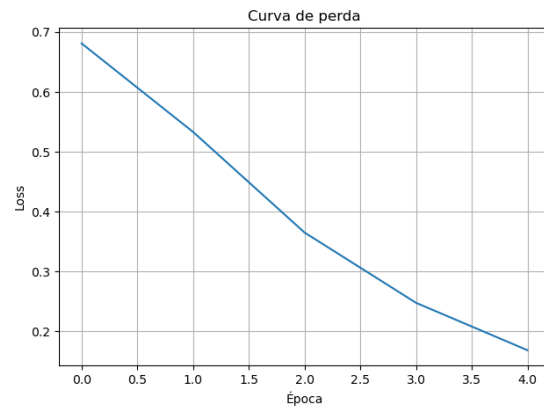
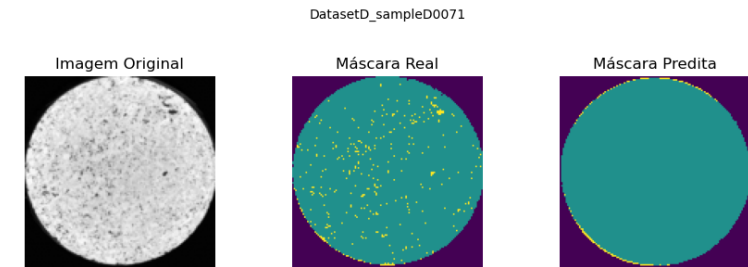
CapsNet (5 épocas)



CKAN (5 épocas)



DeepONet (5 épocas)



Conclusões e próximos Passos

- ☒ • Projeto escalável e com reprodutividade
- ☒ • Dataset com rastreabilidade por nome
- ☒ • Implementação de CapsNet
- ☒ • Implementação de CKAN
- ☒ • Implementação de DeepONet
- ☐ • Implementação do modelo baseline (U-net)
- ☒ • Métricas (Dice, IoU)
- ☐ • Generalização do loop de treino arquivo main_*.py
- ☐ • Plotar curva loss de validação
- ☐ • Salvar as previsões e métricas de teste

Referências

- Convolutional Deep Operator Networks for Learning Nonlinear Focused Ultrasound Wave Propagation in Heterogeneous Spinal Cord Anatomy” - Avisha Kumar, Xuzhe Zhi, Zan Ahmad, Minglang Yin, Amir Manbachi.
<https://arxiv.org/abs/2412.16118>